

sd08. Edge-Device Health Monitoring @ Millions

First-hand 1p3a P0 (bank #8): "Design a central system to monitor the health of edge devices / VMs at millions scale. Compare pull vs push; accepted answer: push primary + pull to double-confirm." Also the bank's own SD entry (VM health monitoring @ 1M). JD tie-in: fleet health is core OCI control-plane work; blast-radius and static-stability vocabulary lands here.

Index

1. Crux & why hard
2. Clarifying questions
3. FR / NFR
4. Back-of-envelope
5. Pull vs push — the graded comparison
6. HLD (push primary + pull verify)
7. Missing-detection (key code)
8. Data model
9. Scaling the collectors
10. Failure modes (incl. network partitions)
11. Ops readiness
12. Trade-offs & defeaters
13. Pop-ups
14. Memorization card

1. Crux & why hard

- Naive: central service pings every device every 30 s → 1M devices = 33k probes/s of pure overhead, probe storms after any blip, and "no answer" is ambiguous (device dead? network? prober overloaded?).
- Minimum primitives: **device-initiated heartbeats** (push), **tiered collectors** (aggregate), **deadline tracking** (who went silent), **verification probe** (pull) before alarming.
- 🗯️ "Push scales because each device does O(1) work; the center only tracks deadlines. Pull remains as the tie-breaker so we page on confirmed-dead, not on one lost packet."

2. Clarifying questions (assumed answers)

- What is a device? → VM / edge box with our agent; can run code
- Detection SLO? → flag within 60 s, page within 2 min, false-page rate low
- What is "health"? → liveness first; extend to metrics (CPU, disk) piggybacked on heartbeats
- Network shape? → devices behind NAT sometimes → another point for push (inbound probes may not even reach)

3. FR / NFR

Req	Target	Enforced by
Detect silent device	< 60 s	15 s heartbeat, miss-3 rule
Confirm before page	probe on suspect	pull verify step
Scale	1M devices, linear	shard collectors by device_id
Central outage ≠ false alarms	static stability	alarm suppression when collector unhealthy
History	status timeline queryable	status change events → store

4. Back-of-envelope

- 1M devices ÷ 15 s = ~67k heartbeats/s; ~200 B each ≈ 13 MB/s — fine over a collector fleet — so what: ingest is easy; the hard part is deadline bookkeeping and alarm quality
- Per-collector 100k devices → 6.7k hb/s each, 10 collectors + headroom — so what: shard by hash(device_id), resharding = moving deadline state (keep it in Redis, not process memory)

5. Pull vs push — the graded comparison

	Pull (center probes)	Push (device heartbeats)
Network	O(N) probes from center; storm risk	O(1) per device; smooth
NAT/firewall	often blocked inbound	outbound almost always works
Ambiguity	no reply = dead or net?	same problem, but cheap to verify
Freshness control	center decides	device config decides (push interval)
Verdict	verification + small fleets	primary channel at scale

- 🔔 "Push 为主, pull double-confirm" — heartbeats drive state; a suspect device gets one targeted probe before we page.

6. HLD

```
[1M devices] agent: heartbeat every 15s {device_id, seq, ts, version, cpu...}
  | [hot path]
  ▼
regional collectors (shard = hash(device_id)) — update last_seen in Redis
  | status-change events only [async]
  ▼
Kafka → status store (Postgres) + alerting + dashboards
  ▲
[verify] prober: on suspect (3 missed), targeted ping/SSH-check → confirm → page
[control plane] device registry (expected fleet, maintenance windows)
```

- Registry is the truth of "who should be reporting" — you can't detect a missing device you don't know about.

7. Missing-detection (key code)

- Don't scan 1M rows per tick; walk time-bucketed deadlines (a poor-man's timer wheel).

```
import time

BUCKET = 5 # seconds

class DeadlineTracker:
    def __init__(self, timeout=45):          # 3 x 15s heartbeats
        self.timeout = timeout
        self.deadline_buckets = {}          # bucket_ts -> set(device_id)
        self.dev_deadline = {}              # device_id -> bucket_ts

    def heartbeat(self, dev, now=None):
        now = now or time.time()
        old = self.dev_deadline.get(dev)
        if old:
            self.deadline_buckets.get(old, set()).discard(dev)
        b = (int(now + self.timeout) // BUCKET) * BUCKET
        self.deadline_buckets.setdefault(b, set()).add(dev)
        self.dev_deadline[dev] = b

    def expired(self, now=None):
        now = now or time.time()
        out = []
        for b in [b for b in self.deadline_buckets if b <= now]:
            out += self.deadline_buckets.pop(b)
        for d in out:
            self.dev_deadline.pop(d, None)
        return out                                # -> SUSPECT, send to prober

# 0(1) per heartbeat; expiry cost proportional to expirations, not fleet size
```

```

class DeadlineTracker {
    private static final long BUCKET = 5, TIMEOUT = 45;
    private final Map<Long, Set<String>> buckets = new ConcurrentHashMap<>();
    private final Map<String, Long> devDeadline = new ConcurrentHashMap<>();

    void heartbeat(String dev, long now) {
        Long old = devDeadline.get(dev);
        if (old != null) buckets.getOrDefault(old, Set.of()).remove(dev);
        long b = ((now + TIMEOUT) / BUCKET) * BUCKET;
        buckets.computeIfAbsent(b, k -> ConcurrentHashMap.newKeySet()).add(dev);
        devDeadline.put(dev, b);
    }

    List<String> expired(long now) {
        List<String> out = new ArrayList<>();
        buckets.keySet().removeIf(b -> {
            if (b > now) return false;
            out.addAll(buckets.get(b));
            return true;
        });
        out.forEach(devDeadline::remove);
        return out;
    }
}
// timer-wheel flavor: expiry work scales with misses, not with 1M devices

```

8. Data model

```

CREATE TABLE devices (
    device_id TEXT PRIMARY KEY,
    region TEXT NOT NULL,
    expected BOOLEAN NOT NULL DEFAULT TRUE, -- registry: should it report?
    maintenance_until TIMESTAMPTZ -- suppress alarms in window
);

CREATE TABLE status_events (
    device_id TEXT REFERENCES devices,
    ts TIMESTAMPTZ NOT NULL,
    status TEXT NOT NULL, -- UP | SUSPECT | CONFIRMED_DOWN | UP_RECOVERED
    source TEXT NOT NULL, -- heartbeat | prober
    PRIMARY KEY (device_id, ts)
);
-- Redis hot state: hb:{device_id} -> last_seen_ts (TTL 2×timeout); status:{device_id}

```

- Store **status changes**, not every heartbeat — 1M×5760/day rows of "still up" is noise; changes are ~nothing.

9. Scaling the collectors

- Shard by `hash(device_id)`; deadline state in Redis per shard → collector restart rebuilds from Redis, no false SUSPECT wave
- Regional collectors keep heartbeat traffic local; only events cross regions
- Tree aggregation for fleet rollups (region → global): dashboards read aggregates

10. Failure modes

- Collector crash → shard's heartbeats fail → suppress: if a whole shard goes silent simultaneously, that's *our* outage, not 100k dead devices — alarm on collector health instead (? classic follow-up)
- Network partition of a region → same shape: mass-SUSPECT + prober also failing → regional "monitoring degraded" alarm, not 100k pages
- Clock skew → use collector receive-time, never device timestamps, for deadlines
- Restart storm (fleet reboot after patch) → heartbeats resume with new seq; UP_RECOVERED events flood → coalesce notifications

11. Ops readiness

- Dashboards: heartbeats/s per shard, SUSPECT rate, confirm-rate (SUSPECT→DOWN %), prober latency
- Alarms: shard silent > 30 s (self), confirm-rate ~100% (real outage) vs ~0% (flaky net), registry-vs-reporting drift
- Runbook: mass SUSPECT → check collector + network first; single DOWN → auto-ticket with last metrics attached

12. Trade-offs & defeaters

- Defeater: "center pings 1M devices" — probe storm, NAT walls; push primary
- Defeater: "page on first missed heartbeat" — UDP-loss pages; miss-3 + pull confirm
- Defeater: "keep deadline state in collector memory only" — restart = false alarm wave; externalize to Redis
- Heartbeat interval: shorter = faster detection but more load; 15 s / miss-3 $\approx$ 45–60 s detection is the standard trade

13. Pop-ups

- ? Piggyback metrics on heartbeats? → yes, version + top-line stats; full metrics go the sd05 pipeline
- ? How does the device know where to send? → bootstrap config + DNS; collectors behind LB (sd06)
- **L5**: cell-ize: collectors per cell, blast radius = one cell; static stability = last-known status served during control-plane outage

14. Memorization card

- Push 15 s heartbeats → sharded collectors → Redis last_seen; miss 3 → SUSPECT → one pull probe → CONFIRMED_DOWN → page

- Timer-wheel deadline buckets: work $\propto$ misses, not fleet
- Store status *changes*; registry defines "expected"; maintenance windows suppress
- Mass-suspect = our outage $\rightarrow$ alarm on ourselves, suppress device pages